ICG:
Introdução ao OpenGL

Profª. Tainá Isabela

Introdução à OpenGL

OpenGL é definida como "um programa de interface para hardware gráfico". Na verdade, OpenGL é uma biblioteca de rotinas gráficas e de modelagem, bi (2D) e tridimensional (3D), extremamente portável e rápida.

Usando OpenGL é possível criar gráficos 3D com uma qualidade visual próxima de um ray tracer.

Introdução à OpenGL

As aplicações OpenGL variam de ferramentas CAD a programas de modelagem usados para criar personagens para o cinema, tal como um dinossauro. Além do desenho de primitivas gráficas, tais como linhas e polígonos, OpenGL dá suporte a iluminação, colorização, mapeamento de textura, transparência, animação, entre muitos outros efeitos especiais.

Atualmente, OpenGL é reconhecida e aceita como um padrão API para desenvolvimento de aplicações gráficas 3D em tempo real.

Introdução à OpenGL

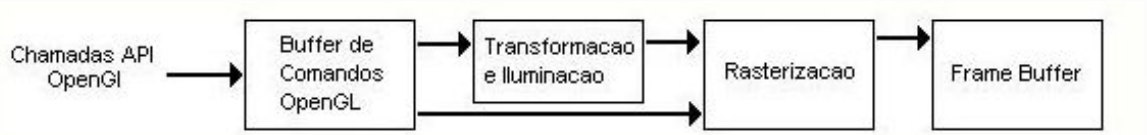
Ao invés de descrever a cena e como ela deve parecer, quando se está utilizando OpenGL é preciso apenas determinar os passos necessários para alcançar a aparência ou efeito desejado. Estes passos envolvem chamadas a esta API portável que inclui aproximadamente 250 comandos e funções.

OpenGL fornece um pequeno conjunto de primitivas gráficas para construção de modelos, tais como pontos, linhas e polígonos. A biblioteca **GLU** (que faz parte da implementação OpenGL) é que fornece várias funções para modelagem, tais como superfícies quádricas, e curvas e superfícies NURBS.

Introdução à OpenGL

A palavra ***pipeline*** é usada para descrever um processo que pode ter dois ou mais passos distintos.

Como uma aplicação faz chamadas às funções API OpenGL, os comandos são colocados em um buffer de comandos. Este buffer é preenchido com comandos, vértices, dados de textura, etc. Quando este buffer é "esvaziado", os comandos e dados são passados para o próximo estágio.



Introdução à OpenGL

Após a etapa de aplicação das transformações geométricas e da iluminação, é feita a rasterização, isto é, é gerada a imagem a partir dos dados geométricos, de cor e textura. A imagem final, então, é colocada no frame buffer, que é a memória do dispositivo gráfico. Isto significa que a imagem é exibida no monitor.

Convenções para os Nomes das Funções

Todos os nomes das funções OpenGL seguem uma convenção que indica de qual biblioteca a função faz parte e, freqüentemente, quantos e que tipos de argumentos a função tem. Todas as funções possuem uma raiz que representa os comandos OpenGL que correspondem às funções. Por exemplo, a função **glColor3f** possui Color como raiz.

O prefixo gl representa a biblioteca gl, e o sufixo 3f significa que a função possui três valores de ponto flutuante como parâmetro. Resumindo, todas as funções OpenGL possuem o seguinte formato:

```
<PrefixoBiblioteca> <ComandoRaiz> <ContadorArgumentosOpcional> <TipoArgumentosOpcional>
```

Convenções para os Nomes das Funções

Variações da função do exemplo anterior, glColor3f, podem receber três valores inteiros como parâmetro (glColor3i), três doubles (glColor3d) e assim por diante. Algumas versões da glColor também recebem quatro argumentos. Neste caso, um dos argumentos é usado para especificar o componente alfa (transparência). Esta convenção de adicionar o número e o tipo dos argumentos facilita a memorização da lista de argumentos.

Bibliotecas

Segundo, OpenGL fornece um conjunto de comandos poderosos, mas primitivos. Portanto, todas as rotinas de desenho de alto nível devem ser elaboradas em função destes comandos. Sendo assim, foram desenvolvidas algumas bibliotecas para simplificar a tarefa de programação.

Bibliotecas

GLU - OpenGL Utility Library: contém várias rotinas que utilizam os comandos OpenGL de baixo nível para executar tarefas como, por exemplo, definir as matrizes para projeção e orientação da visualização, e fazer o rendering de uma superfície. Esta biblioteca é fornecida como parte de cada implementação de OpenGL, e suas funções usam o prefixo glu.

Bibliotecas

GLUT - OpenGL Utility Toolkit: é um toolkit independente de plataforma, que inclui alguns elementos GUI (Graphical User Interface), tais como menus pop-up e suporte para joystick. O seu principal objetivo é esconder a complexidade das APIs dos diferentes sistemas de janelas. As funções desta biblioteca usam o prefixo glut.

É interessante comentar que a GLUT substitiu a GLAUX, uma biblioteca auxiliar OpenGL que havia sido criada para facilitar o aprendizado e a elaboração de programas OpenGL independente do ambiente de programação.

Bibliotecas

GLFW - OpenGL Extension to the X Window System: GLFW é uma biblioteca para uso com OpenGL. Seu nome é um acrônimo para Graphics Library Framework e permite que programadores possam criar e gerenciar janelas e contextos OpenGL, assim como interagir com joystick, mouse e teclado.

Um motivo para que bibliotecas como a GLFW sejam necessárias é que a OpenGL por si só não fornece mecanismos para criar o contexto necessário e gerenciar janelas, entradas do usuário, tempo, etc.

Máquina de Estados

OpenGL é uma máquina de estados. É possível colocá-la em vários estados (ou modos) que não são alterados, a menos que uma função seja chamada para isto.

Por exemplo, a cor corrente é uma variável de estado que pode ser definida como branco. Todos os objetos, então, são desenhados com a cor branca, até o momento em que outra cor corrente é especificada.

Máquina de Estados

OpenGL mantém uma série de variáveis de estado, tais como estilo (ou padrão) de uma linha, posições e características das luzes, e propriedades do material dos objetos que estão sendo desenhados.

Muitas delas referem-se a modos que podem ser habilitados ou desabilitados com os comandos glEnable() e glDisable().

Máquina de Estados

Cada variável de estado possui um valor inicial (default) que pode ser alterado. As funções que utilizadas para saber o seu valor são: glGetBooleanv(), glGetDoublev(), glGetFloatv, glGetIntegerv, glGetPointerv() ou glIsEnabled().

Dependendo do tipo de dado, é possível saber qual destas funções deve ser usada.

```python
import glfw
from OpenGL.GL import *

def main():
    if not glfw.init():
        return

    window = glfw.create_window(800, 600, "Triangulo", None, None)

    if not window:
        glfw.terminate()
        return

    glfw.make_context_current(window)

    while not glfw.window_should_close(window):
        glClear(GL_COLOR_BUFFER_BIT)

        glBegin(GL_TRIANGLES)
        glVertex2f(-0.5, -0.5)
        glVertex2f(0.5, -0.5)
        glVertex2f(0.0, 0.5)
        glEnd()

        glfw.swap_buffers(window)
        glfw.poll_events()

    glfw.terminate()

main()
```

Inicialização e Janela (Biblioteca GLFW)

A OpenGL sozinha não consegue criar janelas ou ler teclado; ela apenas desenha. Por isso usamos a GLFW.

glfw.init(): Inicializa a biblioteca GLFW. É o comando que prepara o sistema operacional para criar janelas e lidar com entradas. Se retornar falso, a biblioteca não pôde ser carregada.

glfw.create_window(800, 600, "Triangulo", None, None): Cria uma janela de 800x600 pixels com o título "Triangulo". Os dois últimos parâmetros são para definir o monitor (para tela cheia) e o compartilhamento de recursos.

Inicialização e Janela (Biblioteca GLFW)

glfw.terminate(): Encerra a GLFW e libera a memória que ela estava usando. É chamado no erro ou ao fechar o programa.

glfw.make_context_current(window): Este comando é crucial. Ele diz à OpenGL que todos os comandos de desenho enviados a partir de agora devem acontecer dentro desta janela específica (window).

O Loop de Renderização

Imagens de computador não são estáticas; elas são redesenhadas dezenas de vezes por segundo.

while not glfw.window_should_close(window): Mantém o programa rodando até que o usuário clique no botão de fechar a janela.

glClear(GL_COLOR_BUFFER_BIT): Limpa a tela. Sem isso, o desenho do frame anterior ficaria "sujo" na tela. O GL_COLOR_BUFFER_BIT indica que queremos limpar as cores (geralmente deixando tudo preto).

O Loop de Renderização

glfw.poll_events(): Verifica se houve algum evento do sistema, como cliques de mouse, redimensionamento de janela ou teclas pressionadas. Se você esquecer isso, a janela parecerá "travada".

Desenhando a Geometria (OpenGL)

Este bloco usa o chamado "Modo Imediato", onde enviamos os dados vértice por vértice para a GPU.

glBegin(GL_TRIANGLES): Avisa a GPU: "Prepare-se, vou começar a enviar coordenadas que devem ser interpretadas como um triângulo".

Desenhando a Geometria (OpenGL)

glVertex2f(x, y): Define um ponto (vértice) no espaço 2D.

- (-0.5, -0.5): Canto inferior esquerdo.
- (0.5, -0.5): Canto inferior direito.
- (0.0, 0.5): Topo central.

glEnd(): Finaliza a definição da forma geométrica. A partir daqui, a GPU processa os pontos e preenche o triângulo na memória.

Double Buffering (Gerenciamento de Frames)

glfw.swap_buffers(window): A computação gráfica moderna usa dois buffers (áreas de memória). Enquanto você desenha em um (o Back Buffer), o monitor mostra o outro (o Front Buffer).

Este comando troca os dois, exibindo instantaneamente o triângulo que acabamos de desenhar e evitando que o usuário veja a imagem sendo "construída" linha por linha.

Double Buffering (Gerenciamento de Frames)

glfw.swap_buffers(window): A computação gráfica moderna usa dois buffers (áreas de memória). Enquanto você desenha em um (o Back Buffer), o monitor mostra o outro (o Front Buffer).

Este comando troca os dois, exibindo instantaneamente o triângulo que acabamos de desenhar e evitando que o usuário veja a imagem sendo "construída" linha por linha.

```python
import glfw
from OpenGL.GL import *
from OpenGL.GLU import *

def desenha():
    glClear(GL_COLOR_BUFFER_BIT)

    glColor3f(1.0, 0.0, 0.0)

    glBegin(GL_QUADS)
    glVertex2i(100, 150)
    glVertex2i(100, 100)

    glColor3f(0.0, 0.0, 1.0)
    glVertex2i(150, 100)
    glVertex2i(150, 150)
    glEnd()

def main():
    if not glfw.init():
        print("Erro ao iniciar GLFW")
        return

    window = glfw.create_window(400, 350, "Mistério", None, None)

    if not window:
        glfw.terminate()
        print("Erro ao criar janela")
        return

    glfw.make_context_current(window)

    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    gluOrtho2D(0, 250, 0, 250)

    while not glfw.window_should_close(window):
        desenha()
        glfw.swap_buffers(window)
        glfw.poll_events()

    glfw.terminate()

main()
```

Configuração de Projeção (A "Câmera")

Diferente do código anterior, onde os valores iam de -1 a 1, aqui definimos nossa própria régua de medição.

glMatrixMode(GL_PROJECTION): Informa à OpenGL que as próximas operações de matriz devem ser aplicadas à Matriz de Projeção. É aqui que definimos como o espaço 3D (ou 2D) é projetado na tela.

Configuração de Projeção (A "Câmera")

glLoadIdentity(): "Limpa" a matriz atual, carregando a matriz identidade. Isso garante que configurações de transformações anteriores não interfiram na nova configuração.

gluOrtho2D(0, 250, 0, 250): Define uma projeção ortográfica bidimensional.
A função mapeia a área de visualização. Aqui, dizemos que a borda esquerda da janela é 0, a direita é 250, a base é 0 e o topo é 250.

Definindo Atributos e Primitivas

glColor3f(R, G, B): Define a cor atual do "estado" da OpenGL.

- 1.0, 0.0, 0.0 é Vermelho puro.
- 0.0, 0.0, 1.0 é Azul puro.

glBegin(GL_QUADS): Prepara a GPU para receber 4 vértices que formarão um quadrilátero (quadrado ou retângulo).

glVertex2i(x, y): Define um vértice usando números inteiros (i).

O Efeito de Gradiente (Interpolação)

O código chama **glColor3f** (Vermelho) antes dos dois primeiros vértices e **glColor3f** (Azul) antes dos dois últimos.

A OpenGL realiza a Interpolação de Cores: ela vai criar um degradê suave entre o vermelho e o azul ao longo da superfície do quadrado.

Estrutura de Fluxo

desenha(): Esta é uma função definida pelo usuário que organiza as chamadas de desenho. É uma boa prática de programação para não poluir o main.

print("Erro ao..."): Adiciona um feedback no console caso o hardware não suporte a versão da OpenGL solicitada ou a janela falhe ao abrir.

Resumo:

Comando	Função Principal
gluOrtho2D	Define a "escala" do desenho (Ex: 0 a 250).
glMatrixMode	Escolhe qual "camada" matemática estamos editando.
GL_QUADS	Desenha formas de 4 lados.
glVertex2i	Posiciona pontos usando números inteiros.
Interpolação	Mistura as cores definidas entre os vértices.

Linhas, Pontos e Polígonos

Com apenas algumas primitivas simples, tais como pontos, linhas e polígonos, é possível criar estruturas complexas. Em outras palavras, objetos e cenas criadas com OpenGL consistem em simples primitivas gráficas que podem ser combinadas de várias maneiras. Portanto, OpenGL fornece ferramentas para desenhar pontos, linhas e polígonos, que são formados por um ou mais vértices.

Neste caso, é necessário passar uma lista de vértices, o que pode ser feito entre duas chamadas de funções OpenGL: ***glBegin()*** e ***glEnd()***

Linhas, Pontos e Polígonos

O argumento passado para glBegin() determina qual objeto será desenhado. No exemplo fornecido por, para desenhar foi usada a seguinte seqüência de comandos:

```
glBegin(GL_POINTS);
        glColor3f(0.0f, 0.0f, 0.0f);
        glVertex2i(100, 50);
        glVertex2i(100, 130);
        glVertex2i(150, 130);
glEnd();
```

Linhas, Pontos e Polígonos

O argumento passado para glBegin() determina qual objeto será desenhado. No exemplo fornecido por, para desenhar três pontos pretos foi usada a seguinte seqüência de comandos:

```
glBegin(GL_POINTS);
        glColor3f(0.0f, 0.0f, 0.0f);
        glVertex2i(100, 50);
        glVertex2i(100, 130);
        glVertex2i(150, 130);
glEnd();
```

Linhas, Pontos e Polígonos

Para desenhar outras primitivas, basta trocar GL_POINTS, que exibe um ponto para cada chamada ao comando glVertex, por:

- **GL_LINES:** exibe uma linha a cada dois comandos glVertex;
- **GL_LINE_STRIP:** exibe uma seqüência de linhas conectando os pontos definidos por glVertex;
- **GL_LINE_LOOP:** exibe uma seqüência de linhas conectando os pontos definidos por glVertex e ao final liga o primeiro como último ponto;

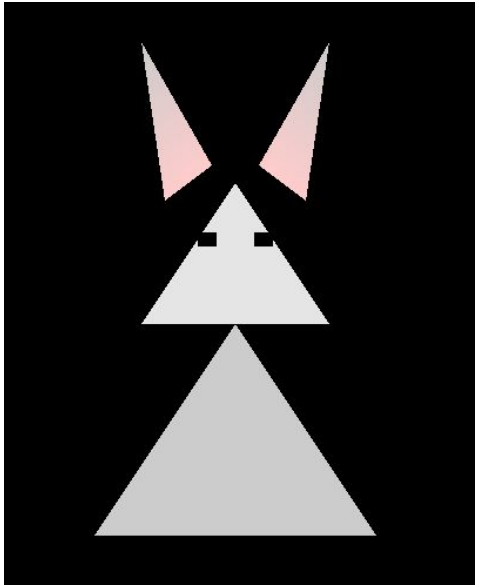
Linhas, Pontos e Polígonos

- **GL_POLYGON:** exibe um polígono convexo preenchido, definido por uma seqüência de chamadas a glVertex;
- **GL_TRIANGLES:** exibe um triângulo preenchido a cada três pontos definidos por glVertex;
- **GL_TRIANGLE_STRIP:** exibe uma seqüência de triângulos baseados no trio de vértices v0, v1, v2, depois, v2, v1, v3, depois, v2, v3, v4 e assim por diante;
- **GL_TRIANGLE_FAN:** exibe uma seqüência de triângulos conectados baseados no trio de vértices v0, v1, v2, depois, v0, v2, v3, depois, v0, v3, v4 e assim por diante;

Linhas, Pontos e Polígonos

- **GL_QUADS:** exibe um quadrado preenchido conectando cada quatro pontos definidos por glVertex;

- **GL_QUAD_STRIP:** exibe uma seqüência de quadriláteros conectados a cada quatro vértices; primeiro v0, v1, v3, v2, depois, v2, v3, v5, v4, depois, v4, v5, v7, v6, e assim por diante.

Questionário

Prática